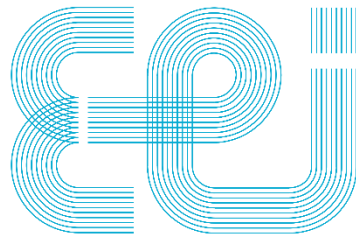


PROGRAMACIÓN TwinCAT

Lab 04

Automatización Industrial
Grado Ingeniería Industrial y Automática



El contenido de esta presentación está basado en los materiales ofrecidos por
Jakob Sagatowski en <https://www.sagatowski.com/>

ÍNDICE

- Funciones
- P00
- Función Bloque
- Librerías

Funciones / Declaración

```
FUNCTION Greater_Smaller
```

```
VAR_INPUT
```

```
    nNumber1 : INT;
```

```
    nNumber2 : INT;
```

```
END_VAR
```

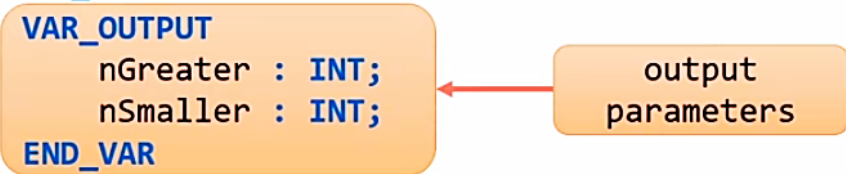
```
VAR_OUTPUT
```

```
    nGreater : INT;
```

```
    nSmaller : INT;
```

```
END_VAR
```

output
parameters



```
IF nNumber1 > nNumber2 THEN
```

```
    nGreater := nNumber1;
```

```
    nSmaller := nNumber2;
```

```
ELSE
```

```
    nGreater := nNumber2;
```

```
    nSmaller := nNumber1;
```

```
END_IF
```

```
PROGRAM MAIN
```

```
VAR
```

```
    nReturnGreater : INT;
```

```
    nReturnSmaller : INT;
```

```
END_VAR
```

```
Greater_Smaller(nNumber1 := 8,  
                nNumber2 := 15,  
                nGreater => nReturnGreater,  
                nSmaller => nReturnSmaller);
```

nReturnGreater

15

nReturnSmaller

8

Funciones / Declaración

FUNCTION SwapNums

VAR_IN_OUT

nNumber1 : INT; 5

nNumber2 : INT; 15

END_VAR

VAR

nNumberTemp : INT;

END_VAR

input & output
parameters at same time!

```
5 nNumberTemp := nNumber1; 5
15 nNumber1 := nNumber2; 15
5 nNumber2 := nNumberTemp; 5
```

PROGRAM MAIN

VAR

nA : INT := 5;

nB : INT := 15;

END_VAR

SwapNums(nNumber1 := nA,
nNumber2 := nB);

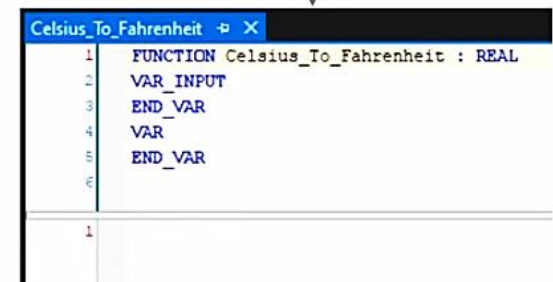
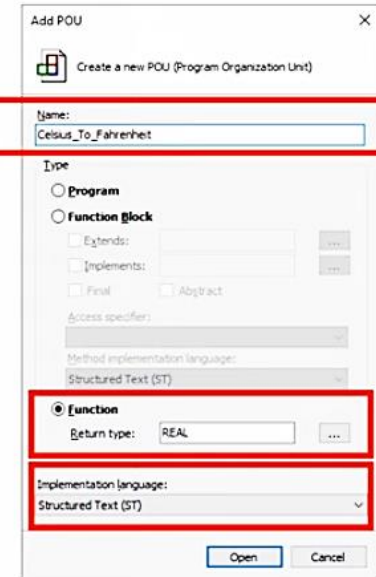
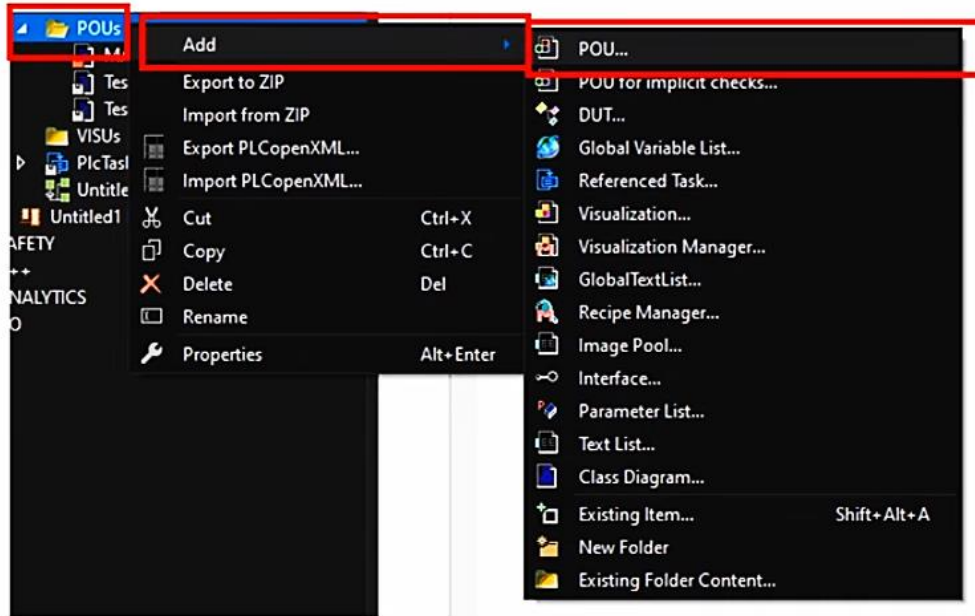
nA

15

nB

5

Funciones / POU

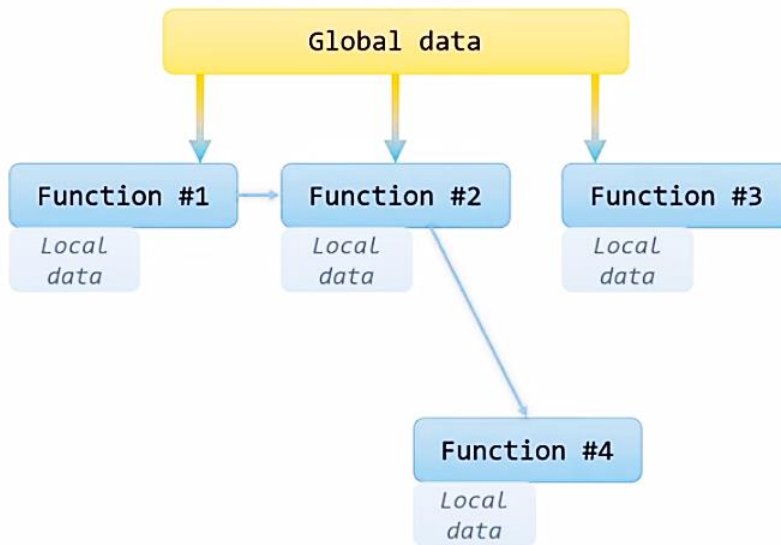


ÍNDICE

- Funciones
- P00
- Función Bloque
- Librerías

Programación Orientada Objetos (POO)

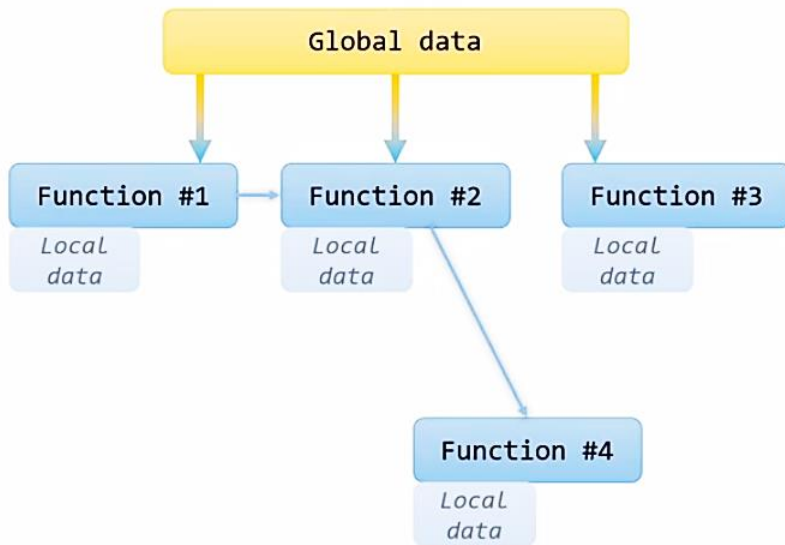
Procedural oriented programming (POP)



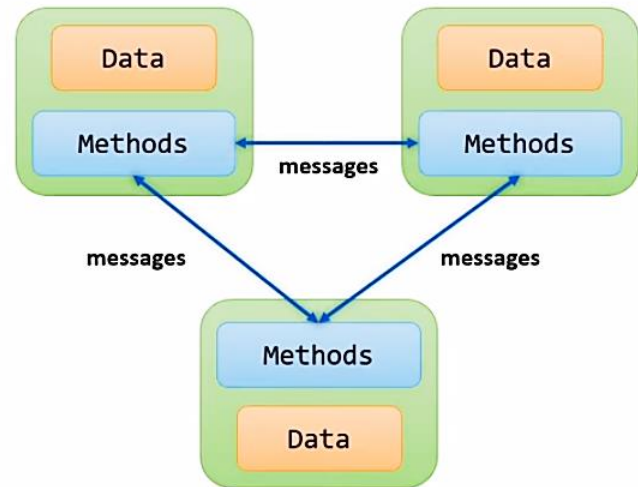
Object oriented programming (OOP)

P00 / Comparativa

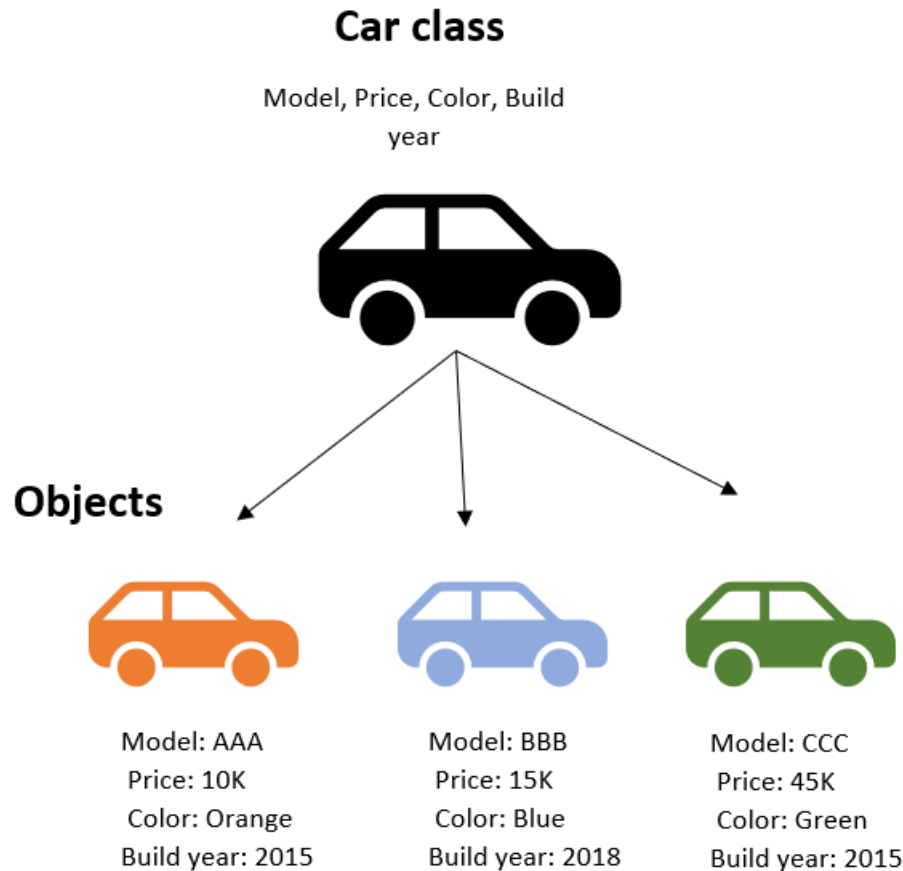
Procedural oriented programming (POP)



Object oriented programming (OOP)



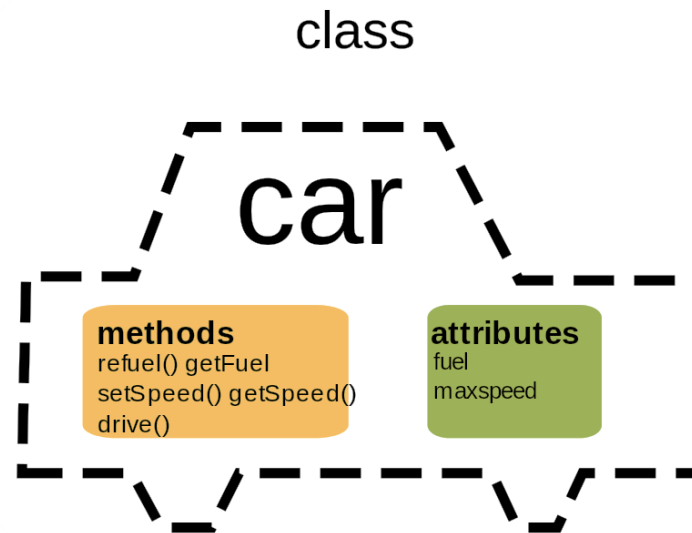
P00 / Clases y Objetos



<https://nonlineardata.com/object-oriented-programming-oop-in-python>

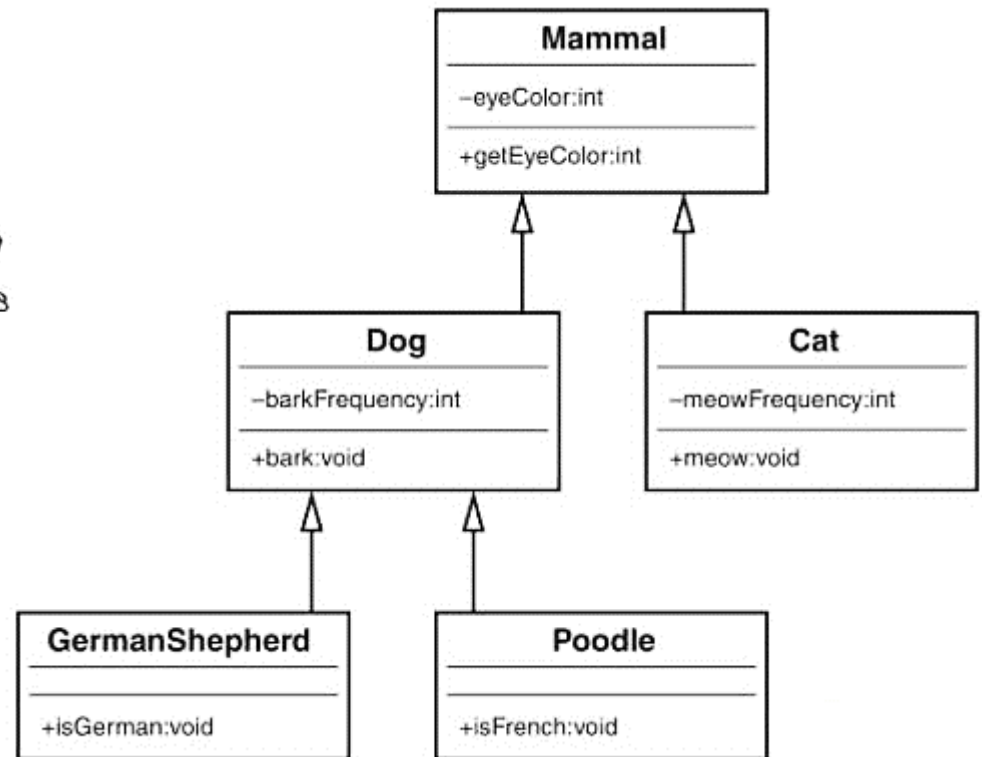
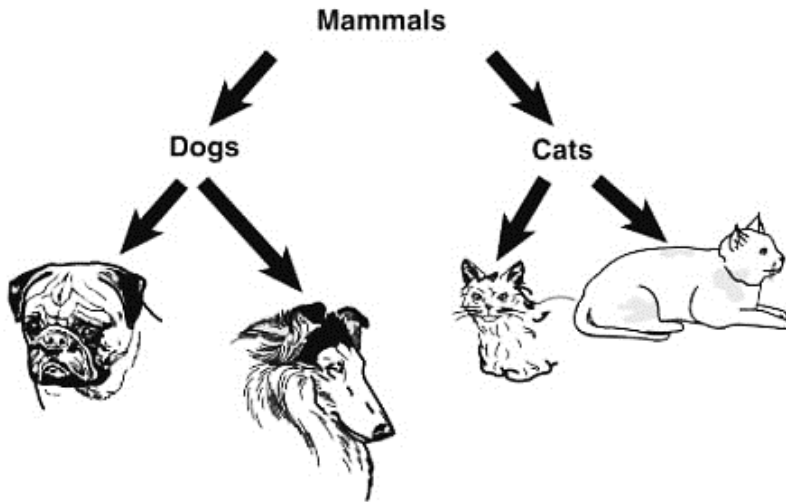
P00 / Atributos y Métodos

- Los **atributos** definen estado objeto.
- Los **métodos** modifican los atributos.



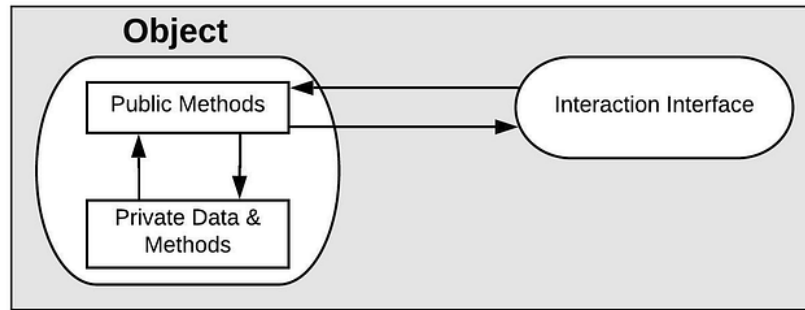
https://en.wikipedia.org/wiki/Attribute_%28computing%29

P00 / Herencia

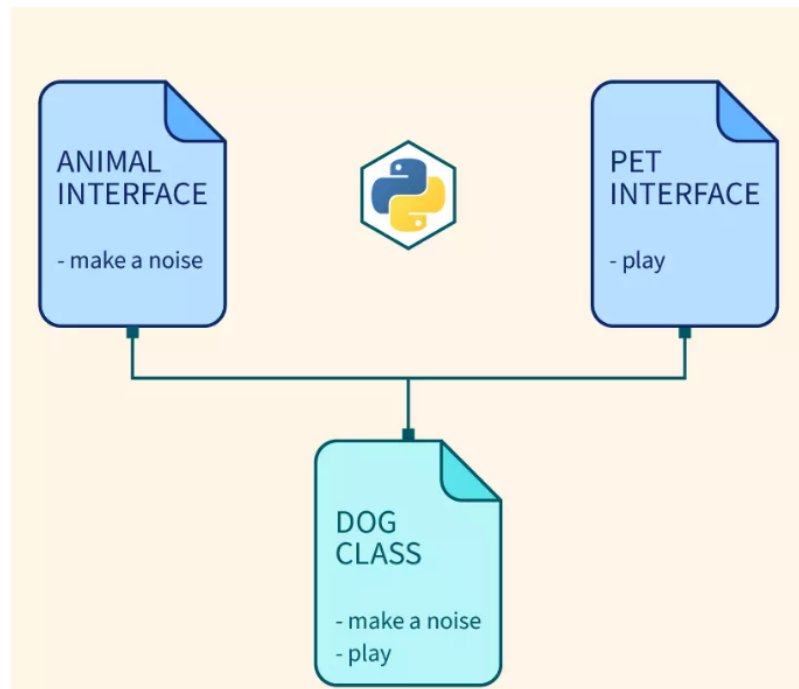


<https://platzi.com/tutoriales/1474-oop/9306-abstraccion-en-poo/>

P00 / Interfaz



<https://levelup.gitconnected.com/a-crash-course-on-object-oriented-programming-in-python-a243b35054c0>



<https://www.scaler.com/topics/interface-in-python/>

P00 / Buenas Prácticas



Single Responsibility Principle

A class should have only a single responsibility (i.e. only one potential change in the software's specification should be able to affect the specification of the class)



Open / Closed Principle

A software module (it can be a class or method) should be open for extension but closed for modification.



Liskov Substitution Principle

Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.



Interface Segregation Principle

Clients should not be forced to depend upon the interfaces that they do not use.



Dependency Inversion Principle

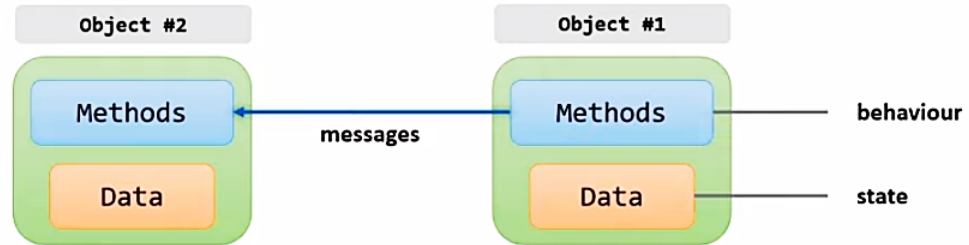
Program to an interface, not to an implementation.

<https://devopedia.org/solid-design-principles>

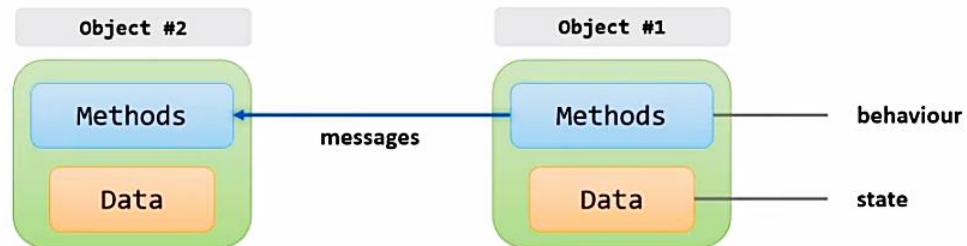
ÍNDICE

- Funciones
- P00
- **Función Bloque**
- Librerías

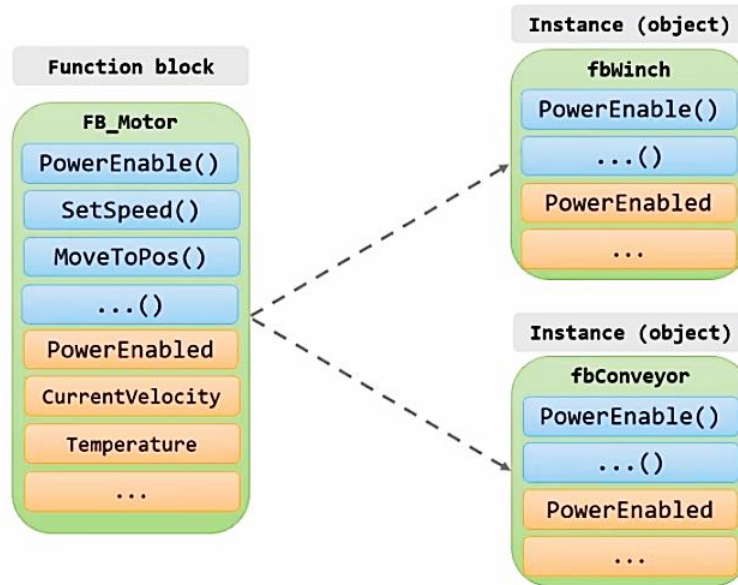
FB / Objetos



FB / Objetos



```
VAR
  fbWinch : FB_Motor;
  fbConveyor : FB_Motor;
END_VAR
```



FB / Definición

Un bloque función es una **clase**

```
FUNCTION_BLOCK FB_Motor
```

```
VAR
```

```
  bPowerOn : BOOL; // [true = on, false = off]
```

```
  fPosition : REAL; // [mm]
```

```
  fTemperature : REAL; // [°C]
```

```
END_VAR
```

internal (instance) variables

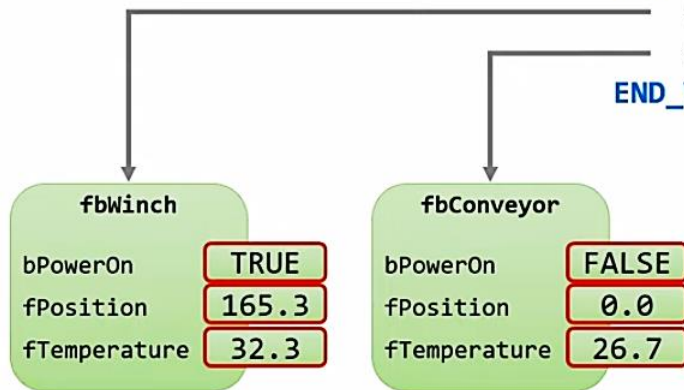
```
PROGRAM MAIN
```

```
VAR
```

```
  fbWinch : FB_Motor;
```

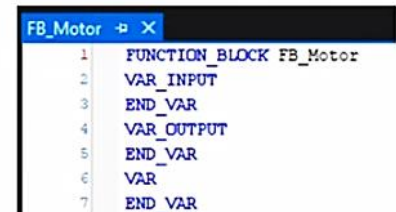
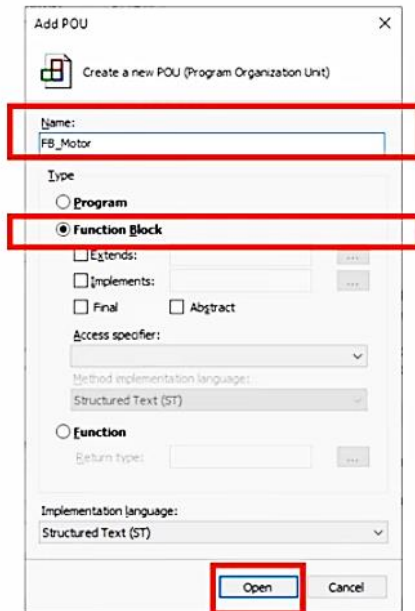
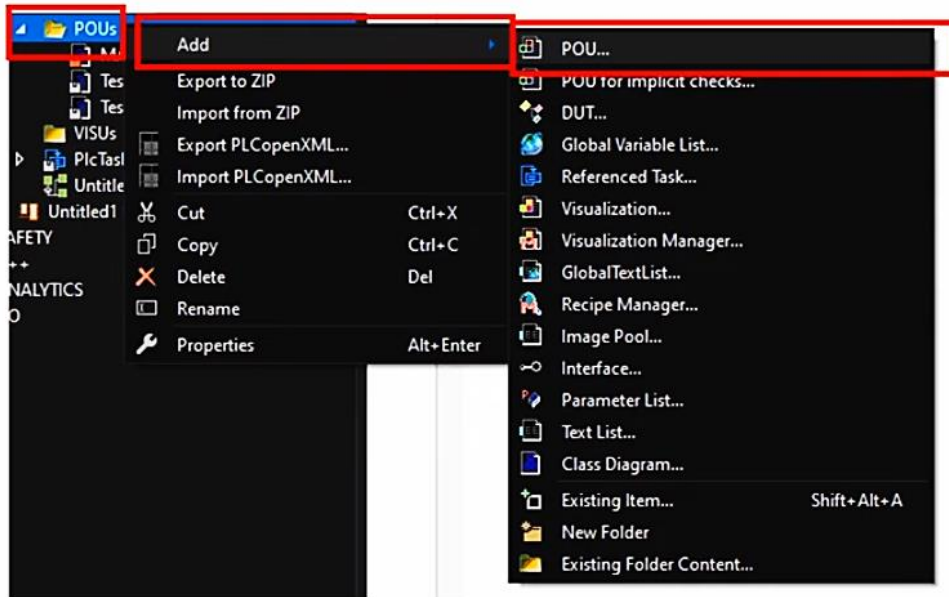
```
  fbConveyor : FB_Motor;
```

```
END_VAR
```



La clase es un molde para crear **objetos**.

FB / POU

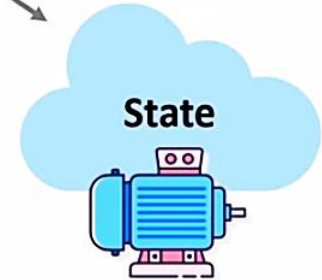


FB / Métodos

FUNCTION_BLOCK FB_Motor

```
METHOD PUBLIC MoveToPosition
VAR_INPUT
    fPosition : REAL; // [mm]
END_VAR
VAR
    // local (scratch) variables
END_VAR
// code

METHOD PUBLIC PowerEnable
VAR_INPUT
    bPowerOn : BOOL;
END_VAR
// code
```

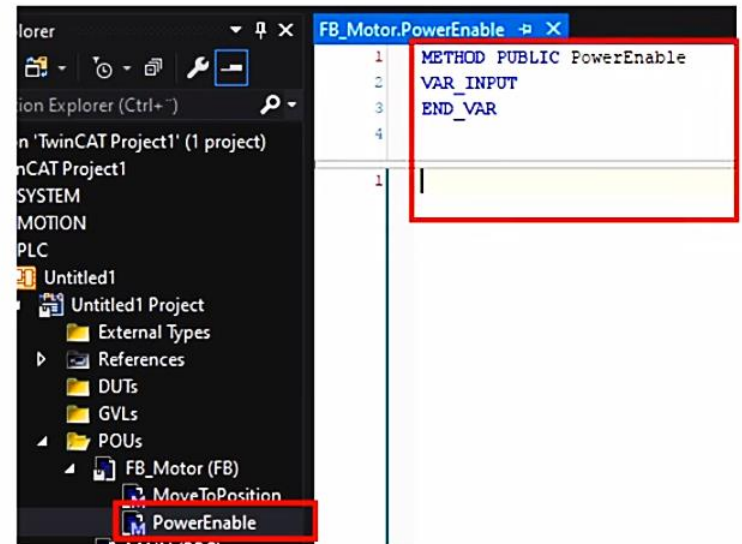
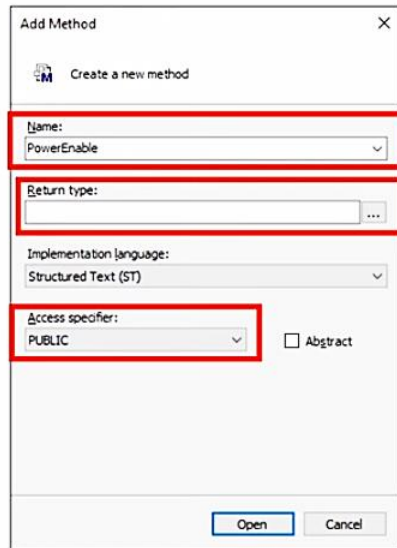
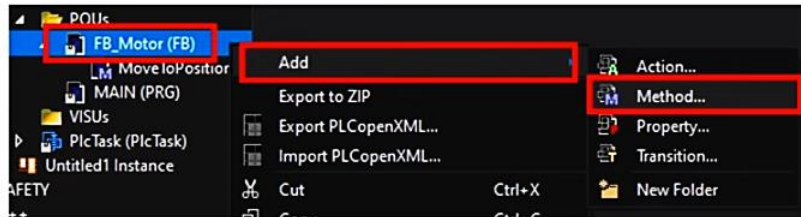


PROGRAM MAIN

```
VAR
    fbWinch : FB_Motor;
END_VAR

fbWinch.PowerEnable(true);
fbWinch.MoveToPosition(42.0);
```

FB / Métodos



FB / Métodos

PUBLIC

No restrictions on accessing methods.

PRIVATE

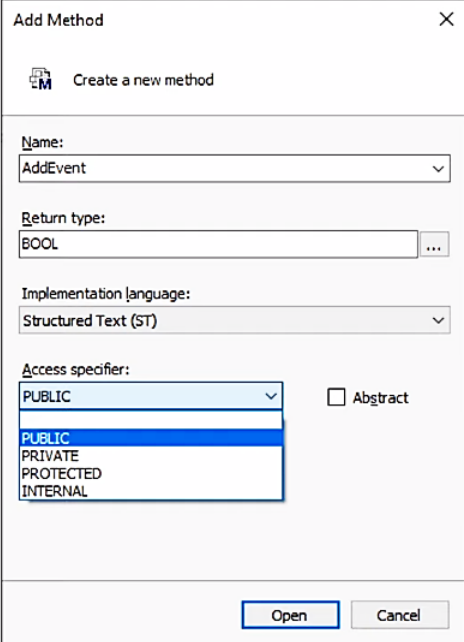
Access is limited to within function block definition.

PROTECTED

Access is limited to within the function block definition and any function block that inherits from the function block.

INTERNAL

Access is limited to the library.

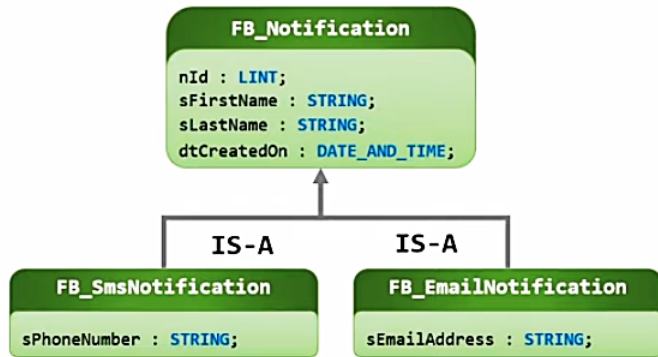


The screenshot shows a dialog box titled "Add Method" with a close button (X) in the top right corner. Below the title bar, there is a button with a document icon and the text "Create a new method". The dialog contains several input fields and a list:

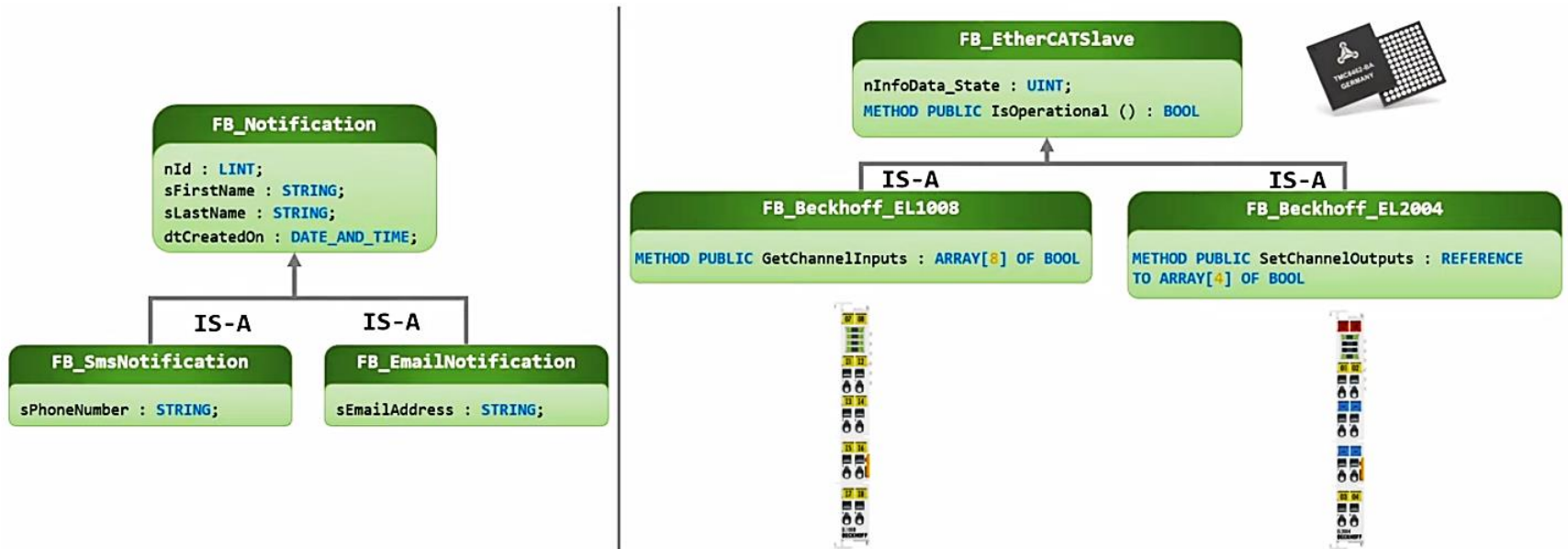
- Name:** A text box containing "AddEvent" with a dropdown arrow on the right.
- Return type:** A text box containing "BOOL" with a dropdown arrow on the right.
- Implementation language:** A dropdown menu showing "Structured Text (ST)".
- Access specifier:** A dropdown menu with a list of options: "PUBLIC", "PRIVATE", "PROTECTED", and "INTERNAL". The "PUBLIC" option is currently selected.
- Abstract:** A checkbox labeled "Abstract" which is currently unchecked.

At the bottom right of the dialog, there are two buttons: "Open" and "Cancel".

FB / Herencia



FB / Herencia



FUNCTION_BLOCK FB_Beckhoff_EL1008 **EXTENDS** FB_EtherCATSlave

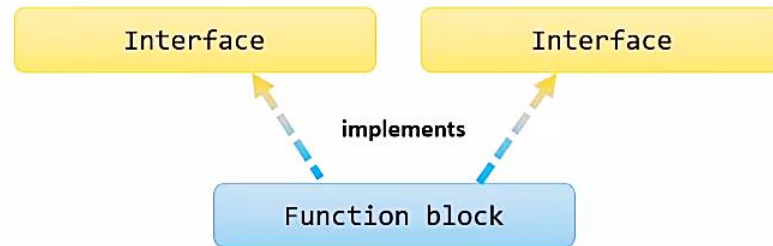
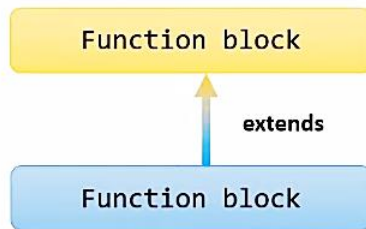
```

PROGRAM MAIN
VAR
    fbEl1008_term1 : FB_Beckhoff_EL1008;
    bIsTerm1_Op : BOOL;
END_VAR

bIsTerm1_Op := fbEl1008_term1.IsOperational();
  
```

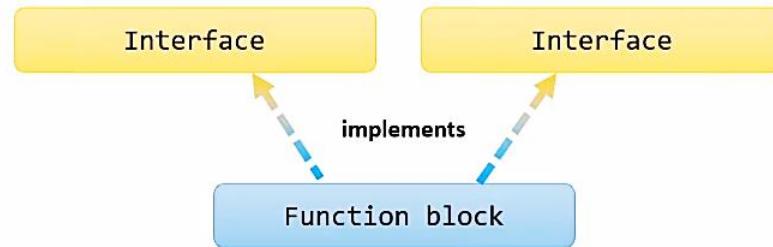
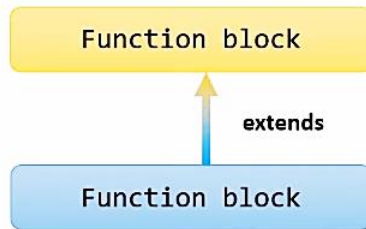
FB / Interfaz

FUNCTION_BLOCK FB_A EXTENDS FB_B

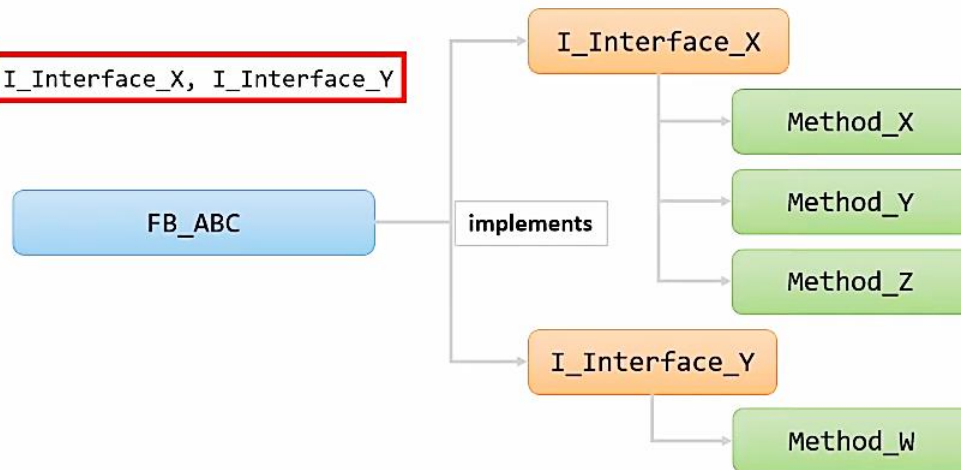


FB / Interfaz

FUNCTION_BLOCK FB_A EXTENDS FB_B



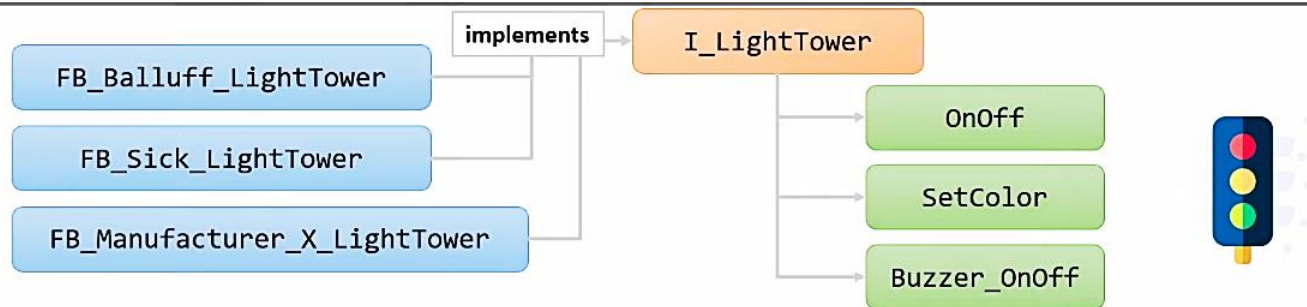
FUNCTION_BLOCK FB_ABC IMPLEMENTS I_Interface_X, I_Interface_Y



FB / Interfaz

```
INTERFACE I_LightTower
    → METHOD OnOff
      VAR_INPUT
        On : BOOL; // [True=Light On]
      END_VAR
    → METHOD SetColor
      VAR_INPUT
        Color : E_Color;
      END_VAR
    → METHOD Buzzer_OnOff
      VAR_INPUT
        On : BOOL; // [True=Buzzer on]
      END_VAR
```

- 1 What to do, not how
- 2 Blueprint of function block
- 3 Programming by contract



FB / Interfaz

INTERFACE I_LightTower

```
METHOD OnOff
VAR_INPUT
    On : BOOL; // [True=Light On]
END_VAR
```

FUNCTION_BLOCK Balluff_LightTower **IMPLEMENTS** I_LightTower

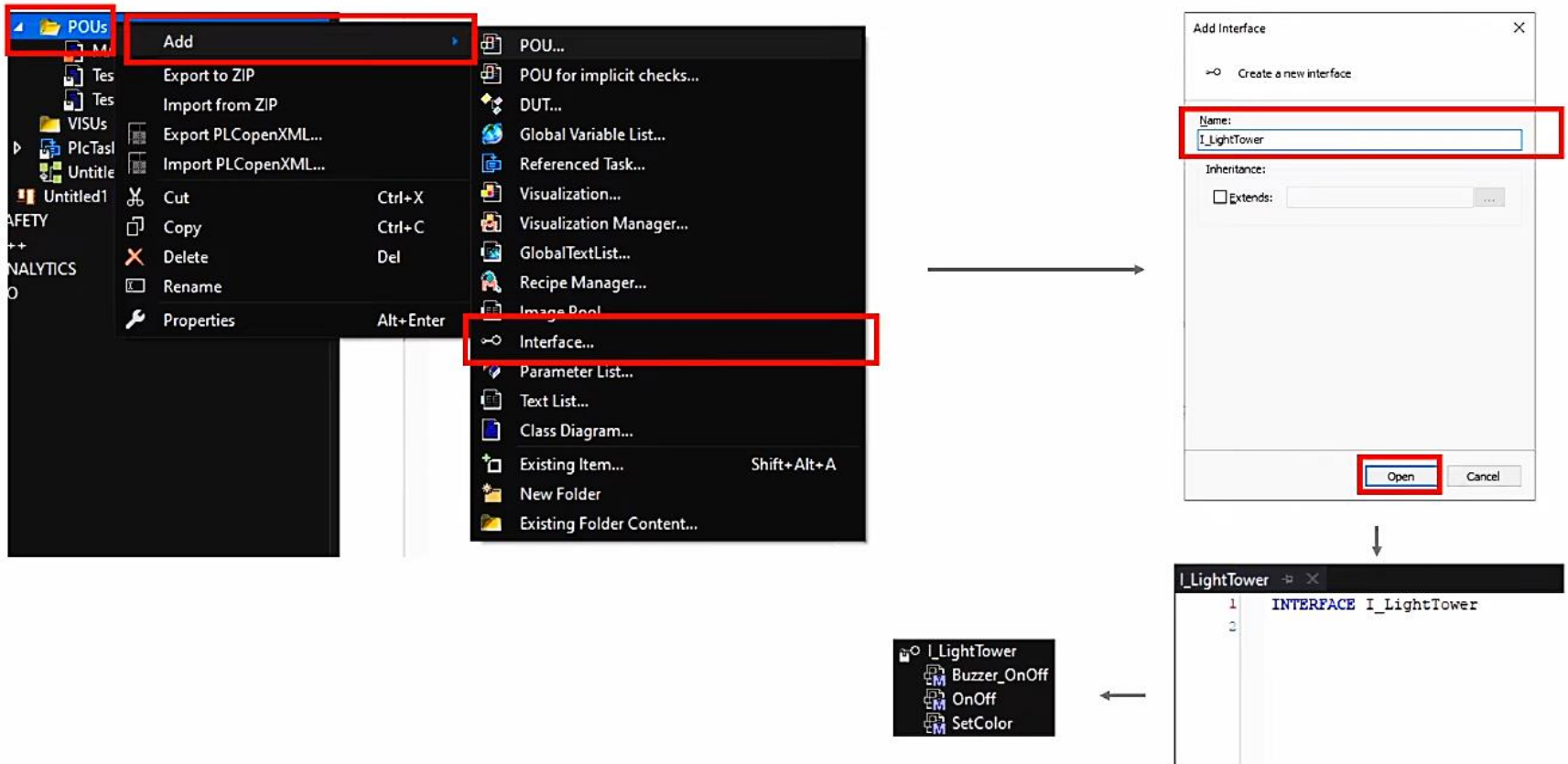
```
METHOD OnOff
VAR_INPUT
    On : BOOL; // [True=Light On]
END_VAR
```

```
// Implementing code for actually turning
// on/off the Balluff light tower, for
// example by using digital outputs or by a
// fieldbus such as EtherCAT
```

```
PROGRAM MAIN
VAR
    fbBalluffLightTower : FB_Balluff_LightTower;
    fbSickLightTower : FB_Sick_LightTower;
    fbLightTower : I_LightTower := fbBalluffLightTower;
END_VAR

fbLightTower := fbSickLightTower;
fbLightTower.On(TRUE);
```

FB / Interfaz



ÍNDICE

- Funciones
- P00
- Función Bloque
- Librerías

Librerías / Básicas

-  Tc2_Math
-  Tc2_EtherCAT
-  Tc2_Uilities
-  Tc2_System
-  Tc2_Standard
-  Tc3_JsonXml

Librerías / TwinCAT 3



TF1xxx – TC3 System



TF5xxx – TC3 Motion



TF2xxx – TC3 Human Machine Interface



TF6xxx – Connectivity



TF3xxx – TC3 Measurement



TF7xxx – Vision

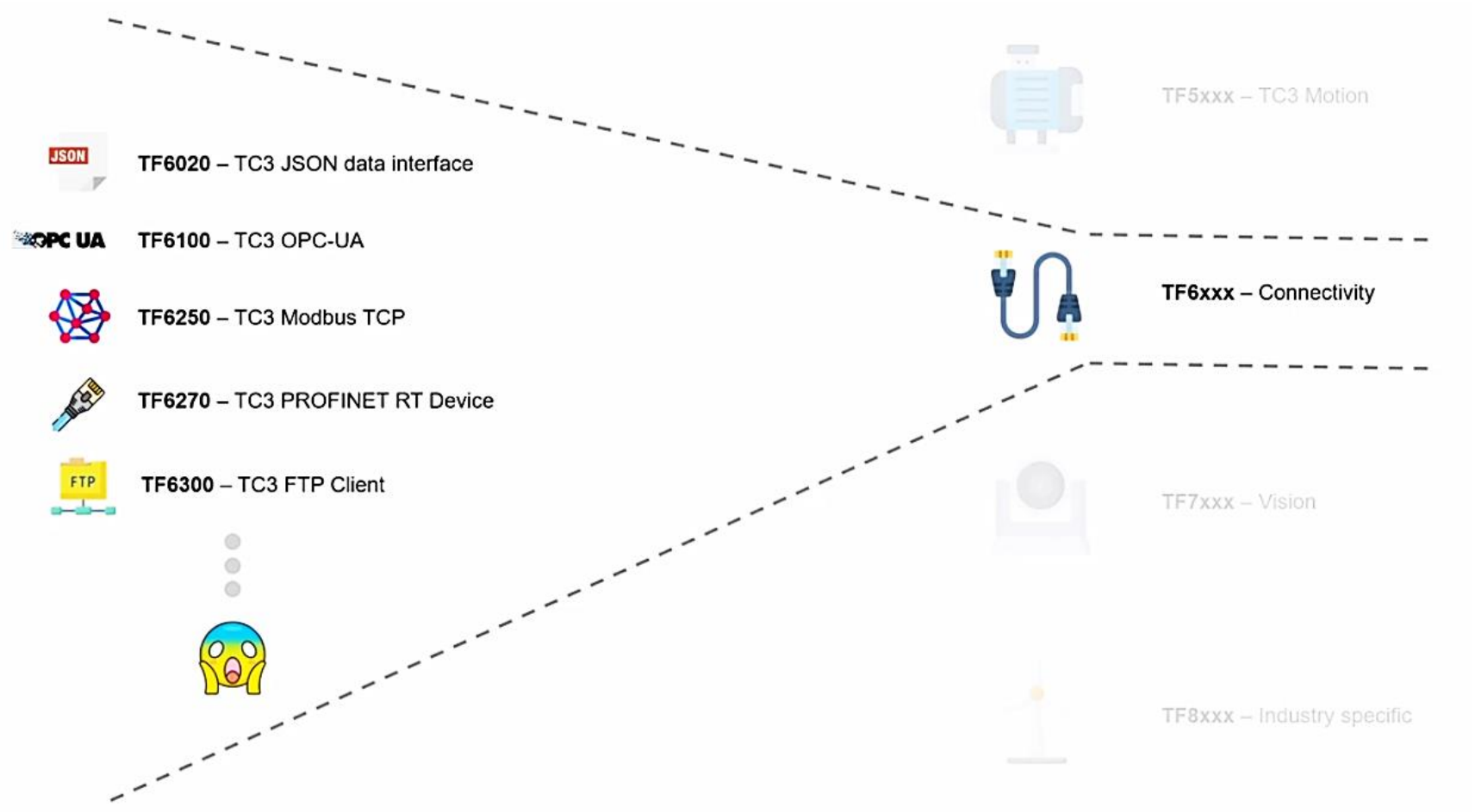


TF4xxx – TC3 Controller

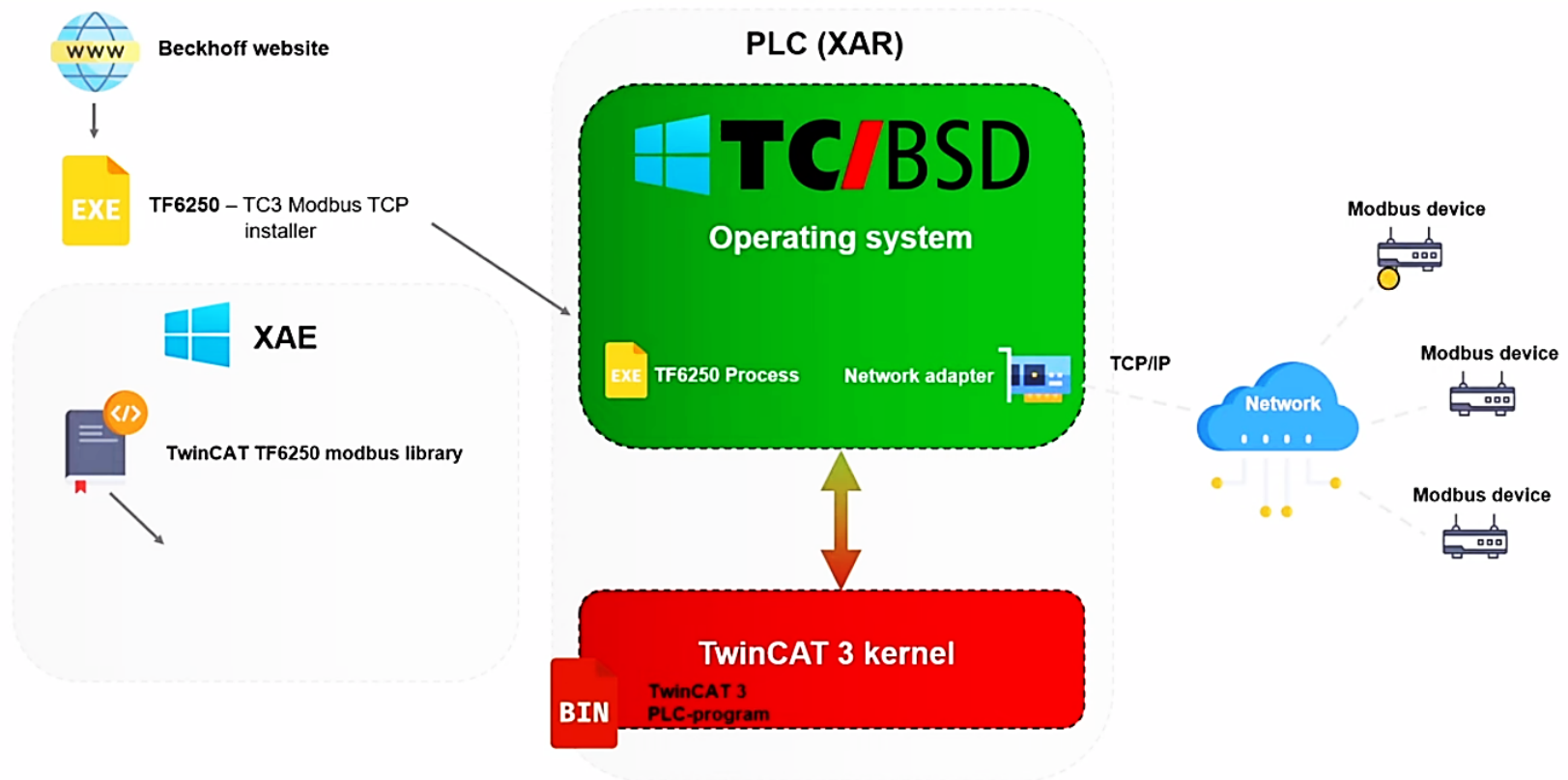


TF8xxx – Industry specific

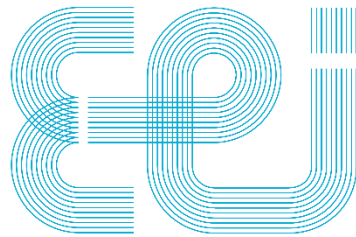
Librerías / TF6250



Librerías / TF6250



¿PREGUNTAS?
andsuarez@uvigo.gal



El contenido de esta presentación está basado en los materiales ofrecidos por
Jakob Satowski en <https://www.sagatowski.com/>